



YISHUN INNOVA JUNIOR COLLEGE
2026 JC1 MOCK PROMOTIONAL EXAMINATION
Higher 2

CANDIDATE
NAME

CG

INDEX NO

COMPUTING

Paper 2 Practical

9569/02

2026

1 hour 50 minutes

Additional Materials:

Removable storage device with the following files:

- `MOCK_PROMO_template.ipynb`
- `RAINFALL.txt`
- `module.py`

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [] at the end of each task.

The total number of marks for this paper is **60**.

Instruction to candidates:

Your program code and output for all the questions should be saved in a single file and downloaded as

PROMO_<your class>_<your name>.ipynb

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1  
Program Code
```

Output:

In [2]:

```
#Task 1.2  
Program Code
```

Output:

In [3]:

```
#Task 1.3  
Program Code
```

Output:

- 1 A school conducts safety inspections on its sports equipment. During each inspection, an inspector records a vibration reading from each item. A reading is a whole number from 0 to 100, inclusive.

At most six items can be inspected in one session.

Task 1.1

Write program code to input and validate:

- the inspector's name, which must not be empty or contain a digit;
- the number of items, which must be from 1 to 6, inclusive; and
- one vibration reading for each item.

Continue requesting a reading until it is valid. Store every valid reading in a list and stop requesting readings when the number stored matches the number of items.

Run your program code using inspector Mira Tan, 5 items and the readings 42, 75, 68, 91 and 54. [4]

A reading of 70 or above means that the item requires attention.

Task 1.2

Using the values obtained in **Task 1.1**, display:

- the inspector's name;
- the list of vibration readings;
- the average vibration reading, rounded to one decimal place; and
- the number of items that require attention. [2]

- 2 The file `RAINFALL.txt` stores five daily rainfall readings for several weather stations. The first line contains the following headers:

```
StationID,Area,D1,D2,D3,D4,D5
```

The next two lines are shown below:

```
R101,North,12.5,0.0,8.0,24.5,5.0  
R102,East,3.0,7.5,7.5,1.0,16.0
```

Task 2.1

Write program code for the function `read_rainfall(filename)` to read `filename` and return a list of tuples in the following format:

```
[
    ("R101", 12.5, 0.0, 8.0, 24.5, 5.0),
    ("R102", 3.0, 7.5, 7.5, 1.0, 16.0),
    # further station records
]
```

The header and the `Area` field must not be included in the returned tuples. [4]

For each station, the rainfall spread is the difference between its highest and lowest daily readings.

Task 2.2

Write program code for the function `summarise_rainfall(stations)` to accept the list returned by **Task 2.1**. Return a list of tuples containing each station ID, its total rainfall and its rainfall spread. Round both calculated values to one decimal place.

For the two records shown above, the returned list begins as follows:

```
[
    ("R101", 50.0, 24.5),
    ("R102", 35.0, 15.0),
    # further station summaries
]
```

[3]

- 3 A shipping company uses seal codes containing uppercase letters and digits. It needs to count how many times a specified character occurs in a seal code.

For example:

```
count_character("A7B7C77", "7") # returns 4
count_character("SEAL", "A")     # returns 1
count_character("SEAL", "X")     # returns 0
```

Task 3

Write recursive program code for the function `count_character(code, target)` to return the number of occurrences of the single character `target` in `code`.

The function must return 0 when `code` is empty. The search is case-sensitive. Do not use iteration or a built-in counting method. [5]

- 4 A shared printer uses round-robin scheduling. Each waiting print job is allowed to print one page before the next job is processed. An unfinished job is returned to the rear of the queue.

Each print job is represented by a tuple in the format:

```
(job_code, pages_remaining)
```

The file `module.py` supplies the following Queue Abstract Data Type (ADT) functions:

- `make_empty_queue()` returns an empty queue;
- `is_empty_queue(queue)` returns `True` if `queue` is empty and `False` otherwise;
- `enqueue(queue, item)` adds `item` to the rear of `queue`; and
- `dequeue(queue)` removes and returns the item at the front of `queue`, or returns `None` if `queue` is empty.

The following code is provided. It creates a queue containing three print jobs in their arrival order.

```
jobs = (("P101", 2), ("P102", 1), ("P103", 3))

print_queue = make_empty_queue()
for job in jobs:
    enqueue(print_queue, job)
```

Task 4.1

Write program code for the function `process_turn(queue)` to process one turn of the round-robin schedule.

The function must:

- return `None` if `queue` is empty;
- call `dequeue()` to obtain the job at the front of `queue`;
- reduce its number of pages remaining by 1;
- if the job has pages remaining, call `enqueue()` to add an updated job tuple to the rear of `queue` and return `None`; and
- if the job has no pages remaining, do not return it to `queue` and return its job code.

Do not use Python list methods to add an item to or remove an item from `queue`. [5]

Task 4.2

Write program code for the function `completion_order(queue)` to:

- repeatedly call `process_turn(queue)` until `queue` is empty;
- store a job code only when `process_turn(queue)` returns one; and
- return a list containing the job codes in the order in which the jobs finish printing.

Test the function using `print_queue`.

The expected result is:

```
['P102', 'P101', 'P103']
```

[3]

- 5 A laboratory stores sample records as tuples in the format `(sample_id, priority)`. A larger priority value means that the sample should be processed earlier.

```
samples = [  
    ("S104", 3),  
    ("S101", 5),  
    ("S106", 2),  
    ("S103", 5),  
    ("S105", 1),  
    ("S102", 4)  
]
```

Task 5

Write program code for the function `bubble_sort(samples)` to:

- perform an in-place bubble sort in descending order of priority;
- swap complete sample tuples when they are in the wrong order;
- count every swap performed; and
- return the number of swaps.

Records with equal priorities must remain in their original relative order. Do not use the built-in `sort()` or `sorted()` functions. [4]

- 6 A hiker records each checkpoint reached on a trail. A Stack Abstract Data Type (ADT), implemented using a Python list, stores the checkpoint codes so that the most recently visited checkpoint is on top.

The file `module.py` supplies the following functions:

- `make_empty_stack()` returns an empty stack; and
- `is_empty_stack(stack)` returns `True` if `stack` is empty and `False` otherwise.

The following tuple contains checkpoints in their visiting order:

```
checkpoints = ("ENTRANCE", "PINE", "RIDGE", "LAKE")
```

Task 6.1

Write program code for the functions:

- `push(stack, checkpoint)` to add checkpoint to the top of stack and return stack; and
- `pop(stack)` to remove and return the top checkpoint, or return `None` if stack is empty.

Create an empty stack named `trail`. Call `push()` to add each item in `checkpoints` to `trail` in the given order. [3]

Task 6.2

Write program code for the function `peek(stack)` to return the top checkpoint without removing it. Return `None` if stack is empty. [2]

When the hiker turns back, the checkpoints are revisited in the reverse of their original visiting order.

Task 6.3

Write program code for the function `backtrack(stack)` to repeatedly call `pop(stack)` until the stack is empty and return a list containing the checkpoint codes in their backtracking order.

Test the function using `trail`. [3]

7 An archive stores its box codes in ascending order:

```
box_codes = [  
    "A104", "A118", "A205", "B107", "B215",  
    "B412", "C103", "C220", "D114", "D305",  
    "E102", "E211", "F109", "F330", "G205"  
]
```

Task 7.1

Write iterative program code for the function `find_box(codes, target)` to perform a binary search for `target` in the sorted list `codes`.

The function must return a tuple containing:

- the index of `target` and the number of comparisons made, if `target` is found; or
- -1 and the number of comparisons made, if `target` is not found.

Count one comparison each time the item at a calculated midpoint is compared with `target`. Do not use a built-in search method. [5]

Task 7.2

Test `find_box()` using:

- "B412", which is present; and
- "D200", which is not present.

Display both returned tuples. [1]

- 8 A parcel service stores parcel records in a hash table. Each record is a tuple in the format `(locker_code, recipient)`. Every locker code is unique.

The table has 11 cells and every unused cell contains `None`. The supplied function `hash_locker(locker_code, table_size)` returns the initial index for a locker code.

When a collision occurs, linear probing with a step of 2 is used. Starting from the initial index, the next index is calculated as follows:

$$\text{next_index} = (\text{current_index} + 2) \% \text{table_size}$$

Continue adding 2 and wrapping around until an unused cell is found. For example, if the initial index is 8, the indices are examined in this order:

8, 10, 1, 3, 5, 7, 9, 0, 2, 4, 6

Since the table size 11 is odd, this process visits every cell once before returning to the initial index.

Task 8.1

Write program code for the function `probe_sequence(initial_index, table_size)` to return a list containing every index in its probing order. Include `initial_index` as the first value and stop before it would be added a second time.

Test the function using `initial_index 8` and `table_size 11`. [2]

The following parcel records are to be inserted:

```
parcels = [  
    ("L204", "Amir"),  
    ("L213", "Beatrice"),  
    ("L222", "Chen Wei"),  
    ("L315", "Divya"),  
    ("L324", "Ethan"),  
    ("L435", "Farah")  
]
```

Task 8.2

Write program code for the function `insert_parcel(table, record)` to:

- obtain the initial index by calling `hash_locker(record[0], len(table))`;
- examine the initial index, followed by each index obtained by adding 2 and wrapping around;
- store `record` in the first cell containing `None` and return `True`; and
- return `False` if every cell has been examined and no unused cell is available.

Create `parcel_table` containing 11 cells, each containing `None`. Insert every record in `parcels` by calling `insert_parcel()`. [4]

Searching must follow the same probing order used for insertion. The search may stop when the matching locker code is found, an unused cell is reached, or every cell has been examined.

Task 8.3

Write program code for the function `find_parcel(table, locker_code)` to return the matching parcel record, or `None` if no matching record is found. Your function must call `hash_locker()` and use linear probing with a step of 2. [2]

9 A wildlife survey stores records as tuples in the format `(species, number_sighted)`.

```
sightings = [  
    ("Hornbill", 18),  
    ("Otter", 7),  
    ("Macaque", 24),  
    ("Kingfisher", 12),  
    ("Monitor Lizard", 7),  
    ("Wild Boar", 15),  
    ("Pangolin", 3),  
    ("Civet", 12)  
]
```

An out-of-place quicksort creates new lists during partitioning. In this question, the first record is used to obtain the pivot value.

Task 9.1

Write program code for the function `partition(records)` to:

- use the `number_sighted` in the first record as the pivot value;
- place every record with a smaller value in a new list named `less`;
- place every record with the same value in a new list named `equal`;
- place every record with a larger value in a new list named `greater`; and
- return `less`, `equal` and `greater`.

The original list `records` must not be changed.

[3]

Task 9.2

Write recursive program code for the function `quick_sort(records)` to return a new list containing the records in ascending order of `number_sighted`.

Your function must:

- return a copy of `records` when it contains at most one item;
- call `partition(records)`;
- recursively sort `less` and `greater`; and
- combine the sorted `less` list, the `equal` list and the sorted `greater` list in the correct order.

Test the function using `sightings`. Do not use the built-in `sort()` or `sorted()` functions.

[5]

END OF PAPER